

Sitewire Users Dashboard

Interview walkthrough notes - Take-home coding challenge

Interview format: Screen share, demo the app, walk through your code, and defend your technical choices. They expect you to understand everything - even if you used AI tools.

1. Demo script (~5 minutes)

Say this while clicking through the app:

1. Dashboard tab - Table appears first; login columns show 'Loading...'
2. Watch rows fill in batch by batch (15 users at a time)
3. Point to the status banner: total user count + 'Loading login data... (X/Y)'
4. Show any failed row - per-row error; other rows still work (partial success)
5. Load log tab - Walk through the event timeline with timestamps
6. Core task + Bonus -> Dashboard - humanized dates, country column, inactive highlighting
7. If asked about code: open hooks/useEnrichedUsers.ts first

2. Challenge recap

- API: <https://fake-users-api.vercel.app>
- ~25% random 500 errors; latency ranges from ~10ms to 3s
- GET /users -> array of users
- GET /users/:id/relationships/logins -> login history per user
- Login arrays are UNSORTED - you must find the most recent login yourself
- Display: user ID, full name, email, last login time, last login IP, total user count
- They care about loading / error / partial-success states, not visual polish

Bonus tasks (you implemented all three)

- Country from last login IP
- Humanized login time ('3 months ago')
- Highlight users inactive for >= 1 month

3. Tech stack

- React 19 + TypeScript + Vite
- date-fns - humanized dates + inactive-month check
- ipwho.is - free IP to country lookup
- Custom hooks only - no Redux / React Query (logic stays visible)

4. File map - where to click in code

hooks/useEnrichedUsers.ts

START HERE. Main orchestration: users -> batched logins -> enrich rows

api/client.ts

fetchWithRetry - 3 retries, 500ms delay on 500/network errors

api/users.ts

getUsers() and getUserLogins(id)

api/geo.ts

getCountryForIp() with in-memory IP cache

utils/login.ts

Most recent login, date formatting, inactive check

hooks/useEventLog.ts

Records load events for the Load log demo tab

components/UserRow.tsx

Per-row loading / success / error UI

components/StatusBanner.tsx

User count, login progress, global error + Retry

components/UserTable.tsx

Table layout; core vs bonus column toggle

types.ts

User, EnrichedUser, LoginStatus

App.tsx

Tabs: Core / Bonus modes; Explain / Dashboard / Load log / Interview prep

5. Data flow - explain this out loud

1. App mounts -> useEnrichedUsers.load() runs automatically
2. GET /users with retry. If all retries fail -> global error banner + Retry button. Table hidden.
3. Users succeed -> table renders immediately. Every row has loginStatus: 'loading'.
4. Batch loop - 15 users at a time. Each batch runs Promise.all on 15 parallel login requests.
5. Per user in batch: fetch logins -> getMostRecentLogin() -> (bonus) geo lookup -> update row.
6. After each batch: loginsLoaded increments -> progress bar and status banner update.
7. Login failure for one user -> that row shows 'Failed to load'. Other rows unaffected.

6. Key technical decisions

Retry logic (api/client.ts)

- Up to 3 retries (4 total attempts) per request
- 500ms delay between attempts
- Retries on HTTP 500 and network errors
- Non-500 errors (e.g. 404) fail immediately - no pointless retries

Batching (size = 15)

- Limits concurrent requests so the flaky API isn't overwhelmed
- Enables progressive UI - rows fill in visibly, not all at once
- Balance: enough parallelism for speed, small enough for incremental feedback

Per-row error handling

- /users failure = global error (can't show the table)
- Single login failure = row-level error only ('Failed to load')
- This is partial success - exactly what the challenge asks for

Stale-load guard (fetchIdRef)

- Each load() increments a counter
- Async callbacks check the counter before updating state
- If user hits Retry mid-load, old in-flight responses are ignored

EnrichedUser type

- Extends User with login fields + loginStatus
- State machine: loading -> success | error
- Keeps row state co-located with user data - easy to render

Country IP cache (api/geo.ts)

- Many users may share the same IP
- Map<ip, country> avoids duplicate geo API calls
- Cache cleared on each full reload
- Geo failure is silent - row shows '-', doesn't break the load

7. Bonus features - how they work

Humanized login time

- `formatDistanceToNow()` from `date-fns` -> '3 months ago'
- title attribute on cell -> exact timestamp on hover/tap
- Core tab shows raw timestamps; Bonus tab shows humanized

Country from IP

- After login IP is fetched, call `https://ipwho.is/{ip}`
- Result cached in memory per IP
- Extra column in bonus mode only

Inactive users (≥ 1 month)

- `differenceInCalendarMonths(now, loginTime) >= 1`
- Only when login loaded successfully and has a timestamp
- Red-tinted row + 'Inactive' badge in `UserRow.tsx`

8. State types (know these)

- `UsersLoadStatus`: loading | error | success - overall /users fetch
- `LoginStatus`: loading | success | error - each row's login fetch
- `EnrichedUser`: User + login fields + `loginStatus` - table row data

9. Questions they might ask

Why batch size 15?

Balance between speed and not hammering a flaky API. Small enough for progressive UI, large enough for parallelism. In production I'd tune with real metrics.

Why not Promise.all for all users at once?

100+ concurrent requests on a 25%-failure API creates a thundering herd and long wait with zero feedback. Batching gives incremental progress.

Why not React Query / SWR?

Small challenge - I wanted to show I understand retry, batching, and state without hiding logic in a library. In production I'd use React Query with per-key retry.

What if /users fails vs one login fails?

/users failure = global error, table hidden, Retry button. Single login failure = only that row shows 'Failed to load'.

How do you find the most recent login?

`reduce()` comparing new `Date(login_time)`. The API does not sort the array.

What happens if the user clicks Retry mid-load?

`fetchIdRef` increments. Any in-flight response from the old load checks the ref and bails out before updating state.

What would you improve with more time?

Exponential backoff, request deduplication, virtualized table, unit tests for retry/batch logic, error boundaries, accessibility audit.

10. API endpoints used

- GET <https://fake-users-api.vercel.app/users>
- GET <https://fake-users-api.vercel.app/users/:id/relationships/logins>
- GET <https://ipwho.is/:ip> (bonus only)

11. App tabs (your built-in notes)

- Core task / Core + Bonus - toggle which features are shown
- Explain - quick summary + live stats
- Dashboard - the actual app to demo
- Load log - event timeline; use 'Copy notes' during demo
- Interview prep - full notes in the app (same content as this PDF)

Good luck tomorrow!